

Trabalho Final

Arquitetura de Computadores

Prof. Pedro Botelho

26º Junho de 2025

1 Descrição

Seu trabalho será implementar um programa (em qualquer linguagem que suporte operações bit-a-bit, preferencialmente C ou C++) que simule a execução de um **processador RISC**, contendo as instruções mostradas na tabela fornecida na Seção 3.

Considere que o processador possui uma palavra de dados e de instrução de 16 bits, e 16 registradores de trabalho (R0-R15), onde R14 é usado como ponteiro de pilha (SP) e R15 é usado como contador de programa (PC). Adicionalmente, o processador possui os registradores de instrução (IR) e de estado (FLAGS).

2 Especificação do Projeto

O simulador deverá receber como entrada um arquivo de texto **contendo** o código a ser executado, em notação hexadecimal (ou Assembly, vide Seção 4). Este arquivo representa a memória de programa, onde cada linha corresponde a um endereço e a uma instrução, no formato <endereço>:<conteúdo>. O preenchimento da memória de dados é **opcional**, podendo ser no mesmo arquivo, após as instruções, ou em arquivos separados.

2.1 Estrutura do Processador

O processador simulado pode usar o modelo de **von Neumann** (dados e instruções na mesma memória) ou **Harvard** (dados e instruções em memórias diferentes), ficando a escolha a cargo do aluno. Ainda, cada posição de memória guarda 16 bits.

O processador possui apenas duas *flags*: **zero** (ativada quando o resultado da ULA é zero) e **carry** (ativada quando o resultado da ULA tem um vai-um do bit 15 para o 16 ativo). De forma a simplificar, a *flag carry* pode ser ativa quando Rn for maior que Rm, ou seja, quando o operando B da ULA for maior que o operando A, o que resulta no mesmo que verificar o bit do vai-um (em comparações sem sinal).

Os saltos do processador são baseados em **comparações sem sinal** (unsigned). Portanto, apenas as *flags* zero (Z) e *carry* (C) serão usadas. Ao usar uma instrução de salto condicional, J<cond>, a condição será avaliada com base nas *flags* de estado. Apenas as instruções da ULA modificam as *flags*.

2.2 Estrutura do Simulador

O programa deve executar até encontrar uma instrução **HALT**. Ao final da execução, o simulador deverá apresentar o conteúdo (em notação hexadecimal, indicada pelo prefixo ‘0x’) dos seguintes componentes:

- **Registradores:** Todos iniciam zerados (R0-R13, PC e PC).
- **Memória de Dados:** Assuma que a faixa de endereços de 0x0000 a 0x8000 está disponível (endereços fora da faixa são lidos como zero) e que a memória está inicialmente zerada.
 - No mesmo formato do arquivo de entrada (<endereço>:<conteúdo>).
 - Devem ser exibidas apenas as posições que forem acessadas pelo código (desconsiderando a pilha).
- **Pilha:** Do tipo descendente (de um endereço maior para um menor) e cheia (SP aponta para o último elemento empilhado).
 - No mesmo formato do arquivo de entrada (<endereço>:<conteúdo>).
 - Assuma que o ponteiro de pilha com o valor inicial de 0x8000 da memória de dados, e que pilha possui também um tamanho de 16 bits.
 - Devem ser exibida apenas se o ponteiro SP não estiver em sua posição original (0x8000).
- **Flags:** Assuma que as todas iniciam zeradas.
 - Exiba os valores finais das *flags carry* e zero.

Sempre que ocorrer uma *pseudoinstrução* **NOP**, o simulador deve exibir estas mesmas informações (veja a Seção 3 para mais informações sobre NOP).

2.3 Sistema de Entrada e Saída (E/S)

O simulador deve fornecer ao usuário um sistema básico de entrada e saída mapeado em memória.

- 0xF000 - Entrada CHAR_IN (simulador lê um caractere da entrada padrão e.g. função *read*)
- 0xF001 - Entrada INT_IN (simulador lê um inteiro da entrada padrão e.g. função *scanf*)
- 0xF002 - Saída CHAR_OUT (simulador escreve um caractere na saída padrão e.g. função *write*)

- 0xF003 - Saída INT_OUT (simulador escreve um inteiro na saída padrão e.g. função *printf*)

Segue abaixo um exemplo em código Assembly que acessa o sistema de E/S:

```

1 MOV R0, #15      // Carrega o endereço de E/S
2 SHL R0, #12
3 ADDI R4, R0, #1  // Obtem o endereço de INT_IN
4 LDR R1, [R4]     // Le um inteiro de INT_IN
5 LDR R2, [R4]     // Le outro inteiro de INT_IN
6 ADD R3, R1, R2   // Soma ambos os inteiros
7 ADDI R4, R0, #3  // Obtem o endereço de INT_IN
8 STR R3, [R4]     // Escreve o resultado em INT_OUT
9 HALT

```

3 Conjunto de Instruções

O processador que será simulado possui 19 instruções simples de 16 bits cada. Os 4 bits superiores das instruções indicam o código da operação (*opcode*). Ainda, os bits 11-10 das instruções J<cond> indicam a condição para o salto, ou seja, o valor das *flags* para que o salto aconteça:

- JMP (*Jump always*): Sempre salta
- JEQ (*Jump if equal*): Salta quando a *flag* zero está ativa
- JNE (*Jump if not equal*): Salta quando a *flag* zero não está ativa
- JLT (*Jump if less than*): Salta quando a *flag* zero não está ativa e a *carry* está
- JGE (*Jump if greater or equal*): Salta quando a *flag* *carry* não está ativa e a zero está

As instruções suportam apenas um modo de endereçamento cada, sendo eles registradores, memória e imediato. Ainda, com 4 bits para registradores, é possível endereçar até 16 registradores (do R0 ao R15). Toda vez que uma instruções é buscada seu PC é incrementado em um, devendo sempre apontar para a próxima instrução antes de ser executado. Abaixo estão as instruções que o processador deverá suportar:

Instrução	Descrição	Tipo	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JMP #Im	PC = PC + #Im	JUMP	0	0	0	0	Im ₁₁	Im ₁₀	Im ₉	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
JEQ #Im	If ¬Z, then PC = PC + #Im	JUMP	0	0	0	1	0	0	Im ₉	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
JNE #Im	If Z, then PC = PC + #Im	JUMP	0	0	0	1	0	1	Im ₉	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
JLT #Im	If ¬Z and C, then PC = PC + #Im	JUMP	0	0	0	1	1	0	Im ₉	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
JGE #Im	If Z or ¬C, then PC = PC + #Im	JUMP	0	0	0	1	1	1	Im ₉	Im ₈	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
LDR Rd, [Rm]	Rd = MEM[Rm]	LOAD	0	0	1	0	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	-	-	-	-
STR Rn, [Rm]	MEM[Rm] = Rn	STORE	0	0	1	1	-	-	-	-	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Rn ₃	Rn ₂	Rn ₁	Rn ₀
MOV Rd, #Im	Rd = #Im	MOVE	0	1	0	0	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Im ₇	Im ₆	Im ₅	Im ₄	Im ₃	Im ₂	Im ₁	Im ₀
ADD Rd, Rm, Rn	Rd = Rm + Rn	ULA	0	1	0	1	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Rn ₃	Rn ₂	Rn ₁	Rn ₀
ADDI Rd, Rm, #Im	Rd = Rm + #Im	ULA	0	1	1	0	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Im ₃	Im ₂	Im ₁	Im ₀
SUB Rd, Rm, Rn	Rd = Rm - Rn	ULA	0	1	1	1	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Rn ₃	Rn ₂	Rn ₁	Rn ₀
SUBI Rd, Rm, #Im	Rd = Rm - #Im	ULA	1	0	0	0	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Im ₃	Im ₂	Im ₁	Im ₀
AND Rd, Rm, Rn	Rd = Rm AND Rn	ULA	1	0	0	1	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Rn ₃	Rn ₂	Rn ₁	Rn ₀
OR Rd, Rm, Rn	Rd = Rm OR Rn	ULA	1	0	1	0	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Rn ₃	Rn ₂	Rn ₁	Rn ₀
SHR Rd, Rm, #Im	Rd = Rm >> #Im	ULA	1	0	1	1	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Im ₃	Im ₂	Im ₁	Im ₀
SHL Rd, Rm, #Im	Rd = Rm << #Im	ULA	1	1	0	0	Rd ₃	Rd ₂	Rd ₁	Rd ₀	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Im ₃	Im ₂	Im ₁	Im ₀
CMP Rm, Rn	Z = (Rm = Rn); C = (Rm < Rn)	ULA	1	1	0	1	-	-	-	-	Rm ₃	Rm ₂	Rm ₁	Rm ₀	Rn ₃	Rn ₂	Rn ₁	Rn ₀
PUSH Rn	SP-; MEM[SP] = Rn	STACK	1	1	1	0	-	-	-	-	-	-	-	-	Rn ₃	Rn ₂	Rn ₁	Rn ₀
POP Rd	Rd = MEM[SP]; SP++	STACK	1	1	1	1	Rd ₃	Rd ₂	Rd ₁	Rd ₀	-	-	-	-	-	-	-	-
HALT	Para a execução	CONTROL	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Como um JMP com deslocamento zero é apenas um salto para a próxima instrução, um JMP #0 deve ser interpretado como uma *pseudoinstrução* **NOP** (*no operation*, ou seja, nada deve ser feito).

4 Avaliação e Entrega

Cada trabalho poderá ser feito em equipes de três, e deverá ser apresentado ao professor, onde será submetido a casos de teste específicos. Serão verificadas as informações mostradas ao final da execução (e sempre que ocorrer uma instrução NOP).

O projeto valerá 12 pontos, avaliado nos seguintes critérios, estando a avaliação de cada critério a cargo do professor.

1. **Simulação:** 8 pontos (Consegue simular a execução de um programa?)
2. **Estrutura e Interface:** 1 ponto (Código foi bem feito e modularizado? Possui uma interface em linha de comando bem feita?)
3. **Sistema de E/S:** 1 ponto (Implementa o sistema de E/S de dados, conforme informa a Seção 2.3?)
4. **Tradução:** 2 pontos extras (Consegue traduzir as instruções em Assembly?)

Conforme se vê acima, o projeto pode receber 2 pontos extras caso, ao invés de receber um código na notação hexadecimal, receber um código em Assembly, devendo o programa o traduzir para que a execução seja possível. Esse código Assembly deve ser capaz de entender etiquetas (*labels*) de código (para saltos). Etiquetas de dados são opcionais.

4.1 Exemplo de Entrada

Seguem abaixo alguns exemplos de entrada em Assembly e seu hexadecimal equivalente:

4.1.1 Exemplo 1

Entrada:

```
1 MOV R0, #3          0000: 0x4003
2 ADDI R2, R0, #2     0001: 0x6202
3 MOV R10, #10        0002: 0x4A0A
4 STR R2, [R0]        0003: 0x3002
5 HALT                0004: 0xFFFF
```

Saída:

```
1 Registradores:
2 R0: 0x0003
3 R1: 0x0000
4 R2: 0x0005
5 R3: 0x0000
6 R4: 0x0000
7 R5: 0x0000
8 R6: 0x0000
9 R7: 0x0000
10 R8: 0x0000
11 R9: 0x0000
12 R10: 0x000A
13 R11: 0x0000
14 R12: 0x0000
15 R13: 0x0000
16 SP: 0x8000
17 PC: 0x0004
18
19 Memoria de Dados:
20 000A: 0x0005
21
22 Flags:
23 Z = 0
24 C = 0
```

4.1.2 Exemplo 2

```
1 main:  MOV R0, #3          0000: 0x4003
2         MOV R1, #5          0001: 0x4105
3 loop:  ADD R2, R1, R0       0002: 0x5210
4         PUSH R2             0003: 0xE002
5         POP R10             0004: 0xFA00
6         SUB R2, R10, R1     0005: 0x72A1
7         CMP R2, R10         0006: 0xD02A
8         JEQ loop           0007: 0x13FA
9         HALT                0008: 0xFFFF
```

```
1 Registradores:
2 R0: 0x0003
```

```
3 R1: 0x0005
4 R2: 0x0003
5 R3: 0x0000
6 R4: 0x0000
7 R5: 0x0000
8 R6: 0x0000
9 R7: 0x0000
10 R8: 0x0000
11 R9: 0x0000
12 R10: 0x0008
13 R11: 0x0000
14 R12: 0x0000
15 R13: 0x0000
16 SP: 0x8000
17 PC: 0x0008
18
19 Flags:
20 Z = 0
21 C = 1
```